



Trabalho 01 de Implementação: **Sistema *New Cross Burger*** (EDL – TADS4M)

1. CONTEXTO DO PROBLEMA

A hamburgueria *New Cross Burger* processa pedidos manualmente em papel, o que causa confusão na ordem de preparo, dificuldade em desfazer erros de digitação no pedido e perda do histórico de vendas do dia. Este trabalho propõe um sistema em linha de comando (ou com interface simples) que resolva esses três problemas usando três estruturas de dados lineares diferentes, cada uma escolhida pela característica de acesso que o problema exige.

2. OBJETIVO GERAL

Implementar um sistema de gerenciamento de pedidos que utilize lista encadeada, pilha e fila, encadeadas manualmente (sem usar coleções prontas da linguagem, como `ArrayList`, `LinkedList`, `Stack`, `Queue` ou `Deque`), demonstrando domínio de ponteiros/referências, alocação dinâmica de nós e operações fundamentais de cada estrutura.

3. ESTRUTURAS EXIGIDAS E JUSTIFICATIVA DE USO

3.1 Lista duplamente encadeada — Histórico de Pedidos do Dia

Guarda todos os pedidos já fechados. É necessário navegar para frente e para trás (consultar o pedido anterior/próximo, buscar por número de pedido, remover um pedido cancelado no meio da lista). Apenas uma lista duplamente encadeada permite deslocamento nos dois sentidos com custo $O(1)$ por passo.

3.2 Fila encadeada simples — Fila de Preparo da Cozinha

Os pedidos devem ser preparados na ordem de chegada (política FIFO — *First In, First Out*). O primeiro pedido feito é o primeiro a ser preparado. O pedido é enfileirado ao ser fechado e desenfileirado quando a cozinha inicia o preparo.

3.3 Pilha encadeada simples — Pilha de Ações (Desfazer) e Pilha de Retirada

Duas aplicações válidas — o grupo pode escolher uma ou implementar as duas:

- Desfazer (undo): cada alteração no pedido em construção (adicionar item, remover item, aplicar desconto) é empilhada; "desfazer" desempilha a última ação.

- Pilha de retirada: pedidos prontos são empilhados na bancada (o último que fica pronto é retirado primeiro, simulando uma bandeja em que só se pode retirar do topo).

4. REQUISITOS FUNCIONAIS MÍNIMOS

Código	Requisito
RF01	Cadastrar um novo pedido (cliente, itens, valor total)
RF02	Adicionar/remover item de um pedido em construção, registrando a ação na pilha de desfazer
RF03	Desfazer a última ação do pedido em construção
RF04	Fechar o pedido: inserir na fila de preparo e na lista de histórico (no fim)
RF05	Chamar a cozinha: desenfileirar o próximo pedido a ser preparado
RF06	Marcar pedido como pronto: empilhar na pilha de retirada
RF07	Retirar pedido pronto: desempilhar o topo
RF08	Consultar histórico navegando para frente e para trás (próximo/anterior)
RF09	Buscar um pedido no histórico por número/cliente
RF10	Cancelar/remover um pedido específico do histórico (remoção no meio da lista)
RF11	Exibir relatório do dia (percorrer a lista do início ao fim, total vendido, quantidade de pedidos)

5. REQUISITOS NÃO FUNCIONAIS

- Implementação própria dos nós (`struct/class Node`) — proibido usar coleções prontas da linguagem para as três estruturas centrais.
- Tratamento de casos extremos: fila/pilha/lista vazia, busca sem resultado, remoção de nó inexistente.
- Menu textual interativo.
- Linguagem de programação C++.

6. MODELAGEM SUGERIDA

```
struct Pedido {
    int numero;
    string cliente;
    string itens[];
    float total;
}

struct NoLista {          // lista duplamente encadeada
    Pedido dado;
    NoLista* anterior;
    NoLista* proximo;
}

struct NoFila {           // fila simplesmente encadeada
    Pedido dado;
    NoFila* proximo;
}

struct NoPilha {          // pilha simplesmente encadeada
    Acao dado;    // ou Pedido, se for pilha de retirada
    NoPilha* proximo;
}
```

7. OPERAÇÕES MÍNIMAS POR ESTRUTURA

Lista duplamente encadeada:

- Inserir no início, no fim e em posição específica
- Remover por valor
- Buscar
- Percorrer para frente
- Percorrer para trás

Fila encadeada:

- Enfileirar
- Desenfileirar
- Verificar se está vazia
- Consultar o primeiro sem remover (peek)

Pilha encadeada:

- Empilhar
- Desempilhar
- Verificar se está vazia
- Consultar o topo sem remover (peek)